

fhq Treap

梁宝烨

佛山市南海区石门中学

2025 年 7 月 13 日

目录

1 引入

2 基本操作

3 区间操作

4 不存储索引的 fhq Treap

5 练习

- 发明人：范浩强神犇；发明项目：fhq treap

- 发明人：范浩强神犇；发明项目：fhq treap
- 要学习 fhq Treap, 你得先简单了解一下普通 Treap 是咋回事

- 发明人：范浩强神犇；发明项目：fhq treap
- 要学习 fhq Treap, 你得先简单了解一下普通 Treap 是咋回事
- Treap 者，Tree 加 Heap 也。它让平衡树上的每一个结点存放两个信息：值和一个随 Treap 者机的索引。其中值满足二叉搜索树的性质，索引满足堆的性质，结合二叉搜索树和二叉堆的性质来使树平衡。这也是 Treap 的性质。

- 发明人：范浩强神犇；发明项目：fhq treap
- 要学习 fhq Treap, 你得先简单了解一下普通 Treap 是咋回事
- Treap 者，Tree 加 Heap 也。它让平衡树上的每一个结点存放两个信息：值和一个随 Treap 者机的索引。其中值满足二叉搜索树的性质，索引满足堆的性质，结合二叉搜索树和二叉堆的性质来使树平衡。这也是 Treap 的性质。
- Treap 为什么可以平衡？我们知道如果对一颗二叉搜索树进行插入的值按次序是有序的，那么二叉搜索树就会退化成一个链表。一个很好的方法就是把插入次序随机化。比如那么我们可以别让数值按次序插入，本来的插入次序是 123456，结果随机之后变成了 352614，树的高度就变小了！

- 当然你也可以选择不理解那么多，你其实只需要知道：

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构
- 还记得二叉搜索树有什么性质嘛？

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构
- 还记得二叉搜索树有什么性质嘛？
- 我们知道如果对一颗二叉搜索树进行插入的值按次序是有序的，

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构
- 还记得二叉搜索树有什么性质嘛？
- 我们知道如果对一颗二叉搜索树进行插入的值按次序是有序的，
- 当前结点左右子树的值都比当前结点值小，反之则一定在右子树上

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构
- 还记得二叉搜索树有什么性质嘛？
- 我们知道如果对一颗二叉搜索树进行插入的值按次序是有序的，
- 当前结点左右子树的值都比当前结点值小，反之则一定在右子树上
- 那么，二叉堆 (大根堆) 的性质是：

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构
- 还记得二叉搜索树有什么性质嘛？
- 我们知道如果对一颗二叉搜索树进行插入的值按次序是有序的，
- 当前结点左子树的值都比当前结点值小，反之则一定在右子树上
- 那么，二叉堆 (大根堆) 的性质是：
- 父结点的**优先级**总是大于或等于任何一个子节点的**优先级**

- 当然你也可以选择不理解那么多，你其实只需要知道：
- 随机大法好，吊打一切卡时间复杂度毒瘤数据
- 可能有同学因为 STL 里有 `priority_queue` 而并没有学过二叉堆 (?) 下面简单说下二叉堆这个基础数据结构
- 还记得二叉搜索树有什么性质嘛？
- 我们知道如果对一颗二叉搜索树进行插入的值按次序是有序的，
- 当前结点左子树的值都比当前结点值小，反之则一定在右子树上
- 那么，二叉堆 (大根堆) 的性质是：
- 父结点的**优先级**总是大于或等于任何一个子节点的**优先级**
- 大家只需要了解上面这一条二叉堆的特性就好

优缺点

- 优点:
- 码量小而却好写，核心操作的代码都是复读机
- 好理解
- 支持的操作多
- 缺点:
- 常数略大

奇怪的操作？

- 普通 Treap 用来维护树平衡的奇怪的操作是什么呢？
- 树旋转
- 不会啊怎么办
- 请出主角:fhq Treap
- fha Treap 的奇怪的操作并且是核心操作只有两个: 分裂 (split) 和合并 (merge)
- 只要掌握好这两个基操，那么你基本上就已经掌握了 fha Treap 了
- 就是把树拆开再拼上拆开再拼上，像拼图一样，很神奇

下面主要搭配代码来讲解。

节点信息

- 我们需要保存这五个信息
- 左右子树编号
- 值
- 索引
- 子树大小

分裂 (split)

- 分裂有两种: 按值分裂和按大小分裂

分裂 (split)

- 分裂有两种: 按值分裂和按大小分裂
- 以下都是我个人的写法, 可以学我的也可以按照自己喜好来

分裂 (split)

- 分裂有两种: 按值分裂和按大小分裂
- 以下都是我个人的写法, 可以学我的也可以按照自己喜好来
- 按值分裂: 把树拆成两棵树, 拆出来的一棵树的值全部小于等于给定的值, 另外一部分的值全部大于给定的值

分裂 (split)

- 分裂有两种: 按值分裂和按大小分裂
- 以下都是我个人的写法, 可以学我的也可以按照自己喜好来
- 按值分裂: 把树拆成两棵树, 拆出来的一棵树的值全部小于等于给定的值, 另外一部分的值全部大于给定的值
- 按大小分裂: 把树拆成两棵树, 拆出来的一棵树等于给定的大小, 剩余部分在另外一棵树

分裂 (split)

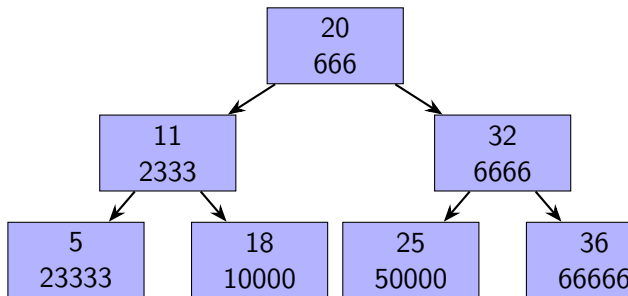
- 分裂有两种: 按值分裂和按大小分裂
- 以下都是我个人的写法, 可以学我的也可以按照自己喜好来
- 按值分裂: 把树拆成两棵树, 拆出来的一棵树的值全部小于等于给定的值, 另外一部分的值全部大于给定的值
- 按大小分裂: 把树拆成两棵树, 拆出来的一棵树等于给定的大小, 剩余部分在另外一棵树
- 一般来说, 我们在使用 fhq Treap 当一棵正常的平衡树用的时候, 使用按值分裂

分裂 (split)

- 分裂有两种: 按值分裂和按大小分裂
- 以下都是我个人的写法, 可以学我的也可以按照自己喜好来
- 按值分裂: 把树拆成两棵树, 拆出来的一棵树的值全部小于等于给定的值, 另外一部分的值全部大于给定的值
- 按大小分裂: 把树拆成两棵树, 拆出来的一棵树等于给定的大小, 剩余部分在另外一棵树
- 一般来说, 我们在使用 fha Treap 当一棵正常的平衡树用的时候, 使用按值分裂
- 而在维护区间信息的时候, 使用按大小分裂。很经典的例子就是文艺平衡树, 这个稍后会说

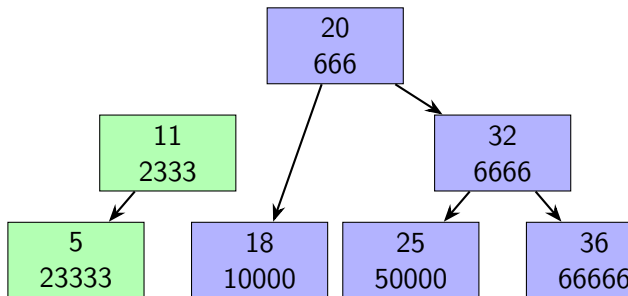
分裂示例

说明：
假如对这个树
按值 17 分裂：



分裂示例

说明：
假如对这个树
按值 17 分裂：



合并 (merge)

- 合并不分两种了，合并就一种
- 合并就是把两颗树 x , y 合并成一棵树，其中 x 树上的所有值都小于等于 y 树上的所有值。并且新合并出来的树依旧满足 Treap 的性质
- 合并就不演示了，上张幻灯片倒放就是合并 (我太懒了)

插入

- 可是这两种操作有啥用啊，我怎么能用这两种操作来完成平衡树的基操呢？

插入

- 可是这两种操作有啥用啊，我怎么能用这两种操作来完成平衡树的基操呢？
- 首先来说插入

插入

- 可是这两种操作有啥用啊，我怎么能用这两种操作来完成平衡树的基操呢？
- 首先来说插入
- 设插入的值为 val ，那么我们的步骤就是：

插入

- 可是这两种操作有啥用啊，我怎么能用这两种操作来完成平衡树的基操呢？
- 首先来说插入
- 设插入的值为 val ，那么我们的步骤就是：
- 按值 val 把树分裂成 x 和 y

插入

- 可是这两种操作有啥用啊，我怎么能用这两种操作来完成平衡树的基操呢？
- 首先来说插入
- 设插入的值为 val ，那么我们的步骤就是：
- 按值 val 把树分裂成 x 和 y
- 合并 x ，新结点， y

- 为什么可以这样呢？

- 为什么可以这样呢？
- 口头描述我们要插入 val 这个值，那么我们按值 val 分裂，于是得到了两棵树 x 和 y

- 为什么可以这样呢？
- 口头描述我们要插入 val 这个值，那么我们按值 val 分裂，于是得到了两棵树 x 和 y
- 按照按值分裂的定义，我们分裂出来的 x 树上的所有值一定都小于等于 val ， y 树上的所有值一定都大于 val

- 为什么可以这样呢？
- 口头描述我们要插入 val 这个值，那么我们按值 val 分裂，于是得到了两棵树 x 和 y
- 按照按值分裂的定义，我们分裂出来的 x 树上的所有值一定都小于等于 val ， y 树上的所有值一定都大于 val
- 那么我们就直接合并 x ，用值 val 新建的新节点， y 就 OK 咯

删除

- 前面的插入很简单，只需要两步

删除

- 前面的插入很简单，只需要两步
- 删除比较麻烦啊，需要四步 (笑)

删除

- 前面的插入很简单，只需要两步
- 删除比较麻烦啊，需要四步 (笑
- 例如我们要删除 `val` 这个值，那么：

删除

- 前面的插入很简单，只需要两步
- 删除比较麻烦啊，需要四步 (笑
- 例如我们要删除 val 这个值，那么：
- 首先，按值 val 把树分裂成 x 和 z

删除

- 前面的插入很简单，只需要两步
- 删除比较麻烦啊，需要四步 (笑
- 例如我们要删除 val 这个值，那么：
- 首先，按值 val 把树分裂成 x 和 z
- 再按值 $val-1$ 把 x 树分裂成 x 和 y

删除

- 前面的插入很简单，只需要两步
- 删除比较麻烦啊，需要四步 (笑
- 例如我们要删除 val 这个值，那么：
- 首先，按值 val 把树分裂成 x 和 z
- 再按值 $val-1$ 把 x 树分裂成 x 和 y
- 那么此时 y 树上的所有值都是等于 val 的，我们去掉它的根节点：让 y 等于合并 y 的左子树和右子树

删除

- 前面的插入很简单，只需要两步
- 删除比较麻烦啊，需要四步 (笑
- 例如我们要删除 val 这个值，那么：
- 首先，按值 val 把树分裂成 x 和 z
- 再按值 $val-1$ 把 x 树分裂成 x 和 y
- 那么此时 y 树上的所有值都是等于 val 的，我们去掉它的根节点：让 y 等于合并 y 的左子树和右子树
- 最后，合并 x, y, z

查询值的排名

- 也很简单，设要查询的值为 val ，那么
- 按值 $val-1$ 分裂成 x 和 y
- 则 x 的大小 $+1$ 就是 val 的排名
- 最后再把 x 和 y 合并起来

前驱/后继

- 设操作数为 val
- 前驱: 按值 $val-1$ 分裂成 x 和 y , 则 x 里面最右的数就是 val 的前驱
- 后继: 按值 val 分裂成 x 和 y , 则 y 里面最左的数就是 val 的后继

一些容易误解的地方

在介绍 fhq_treap “ ”

事实上，对于维护一个序列，只需要把“值”当做下标来看待即可，我们再另外存储对应位置上真正的数是什么

但其实没必要这么拐弯抹角，在区间意义下的按照“值”来分裂，本质上是相当于按照大小来分裂

你也可以理解在维护序列区间的意义下，二叉搜索树有了“新的定义”

- 例如，我们要对区间 $[l, r]$ 操作，那么需要在 fhqtreap 上把它分裂出来，进行操作后再合并回去
- 具体就是按照大小 $l-1$ 分裂出两棵树 x 和 y
- 然后在 y 中按照大小 $r-l+1$ 分裂出两棵树 y' 和 z
- y' 就是我们要操作的区间对应的平衡树
- 操作完记得按照 x, y', z 的顺序合并
- 关于区间操作
- 对于区间加减乘除，可以类比线段树，打懒标记即可
- 而对于文艺平衡树来说，区间翻转也是可以打标记的！

不存储索引的 fhq Treap

- fha Treap 的结点里也可以不存储索引值
- 纵览整个 fha Treap, 维护堆特性的代码其实只有 merge 函数里的一句
- 而且我们知道 fha Treap 的堆维护的是随机的索引值所以我们可以不存放索引
- 而是把这句改成其它的让树随机合并的语句
- 例如 `rand()&1`, 但是这句话的性能不好
- 一些其它写法还有比如把 y 合并到 x 的几率为 $\frac{size(x)}{size(x) + (size(y))}$ 等等
- 可以参考我当年看的上古帖子:
<https://www.luogu.com.cn/discuss/123169>

- P3369 【模板】普通平衡树
- P5338 [TJOI2019] 甲苯先生的滚榜
- P2596 [ZJOI2006] 书架
- P4309 [TJOI2013] 最长上升子序列
- P3391 【模板】文艺平衡树
- P1486 [NOI2004] 郁闷的出纳员